



**Universidad Nacional Autónoma de
México**

Facultad de Ingeniería



Recursos Distribuidos

Semestre 2027 - 1

Laboratorio S5. Mexi Banco v05.1

Alumno: Fernández Herrera Mauricio

Profesor: ING. OSCAR MANUEL RUIZ HURTADO

Grupo: 02

Entrega: 27/09/2026

1.- Clonación del Repositorio.

Clonamos el repositorio de Mexi Banco usando la versión 5.1. Ingresamos al directorio del proyecto y verificamos que el repositorio se encuentra disponible localmente.

```
PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1> git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
Cloning into 'mexi-banco'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 123 (delta 23), reused 107 (delta 12), pack-reused 0 (from 0)
Receiving objects: 100% (123/123), 46.71 KiB | 525.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.
warning: refs/tags/v05.1 cd46b8b91a09070f8262c3b8acfb17caf2405cda is not a commit!
Note: switching to '88203161aa31a371823715684a548d154e1516f8'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1> cd mexi-banco
PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1\mexi-banco> git status
Not currently on any branch
nothing to commit, working tree clean
PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1\mexi-banco> git branch --show-current
PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1\mexi-banco>
```

Figura 1. Clonación y verificación del repositorio Mexi Banco.

2.- Levantamiento de los servicios con Docker Compose

```
PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1\mexi-banco> docker compose ps
NAME                IMAGE              COMMAND                  SERVICE  CREATED      STATUS      PORTS
mexi-banco-app-1    mexi-banco-app    "java -jar app.jar"      app      43 seconds ago Up 37 seconds (healthy)  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine "docker-entrypoint.s..." db       43 seconds ago Up 42 seconds (healthy)  5432/tcp
PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1\mexi-banco> docker compose ps
```

Figura 2. Estado de los servicios app y db después de iniciar Docker Compose.

Ejecutamos el comando `docker compose up --build -d` para construir la imagen de la aplicación y levantar los servicios definidos en el archivo de Docker Compose. Durante el proceso se creó la red utilizada por los contenedores y un volumen para el almacenamiento persistente de los datos de la base de datos.

La construcción de la imagen de la aplicación finalizó correctamente. Posteriormente, mediante `docker compose ps`, se verificó que tanto el contenedor de la aplicación como el de PostgreSQL se encontraran en estado `healthy`. Además, se comprobó que la aplicación quedó disponible mediante el puerto 8080 del equipo anfitrión.

3.- Prueba de los endpoints mediante Postman

Comprobamos el funcionamiento de la aplicación importándola en Postman. Posteriormente, ejecutamos las diferentes solicitudes HTTP incluidas en la colección, verificando las respuestas obtenidas por parte del servicio.

Las pruebas permitieron comprobar el funcionamiento de los distintos endpoints de Mexi Banco, incluyendo operaciones realizadas correctamente y respuestas correspondientes a situaciones de error. En cada caso se verificó tanto el código de estado HTTP como la información devuelta en el cuerpo de la respuesta.

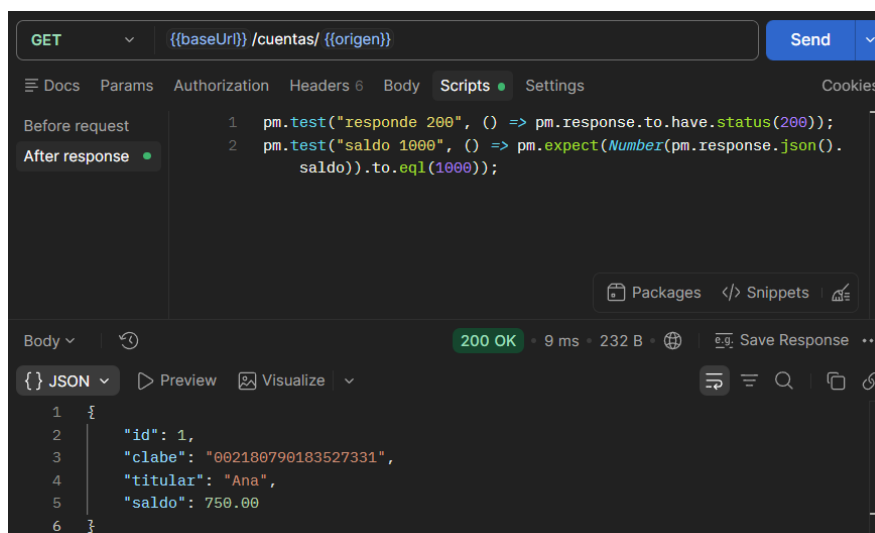


Figura 3. Prueba de consulta de una cuenta.

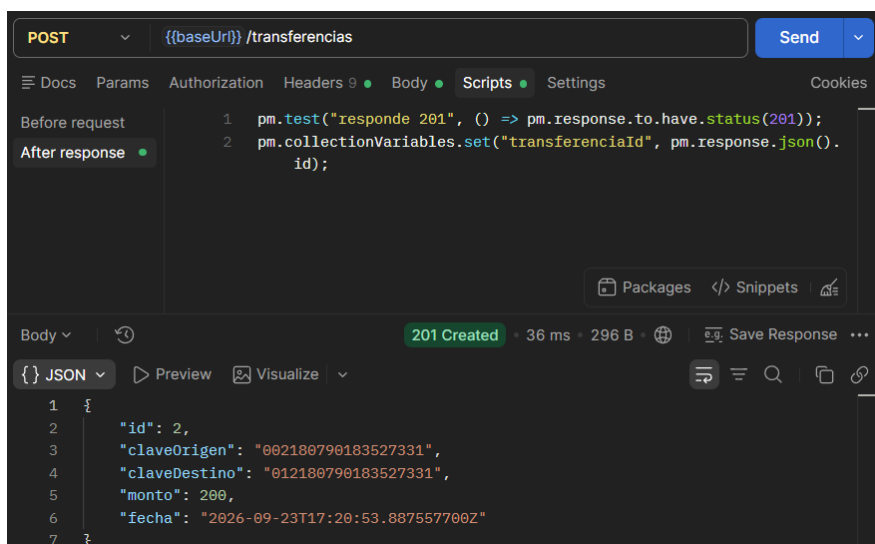


Figura 4. Prueba de transferencia entre cuentas.

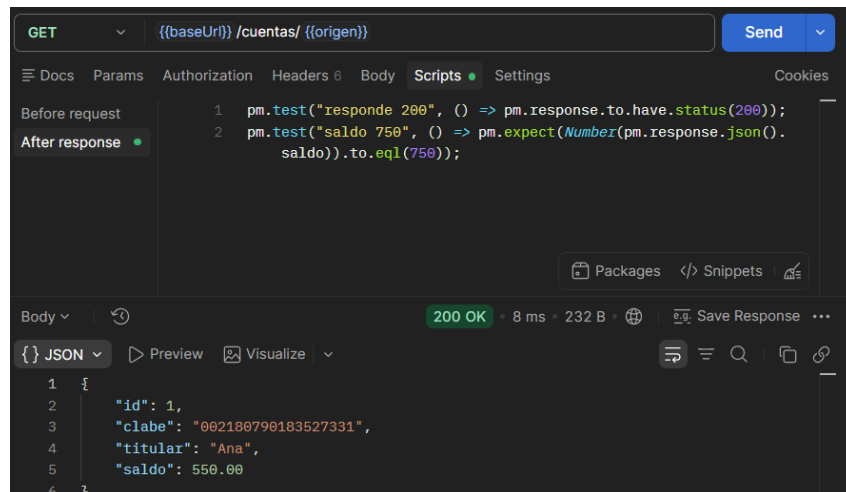


Figura 5. Consulta tras operaciones de SPEI.

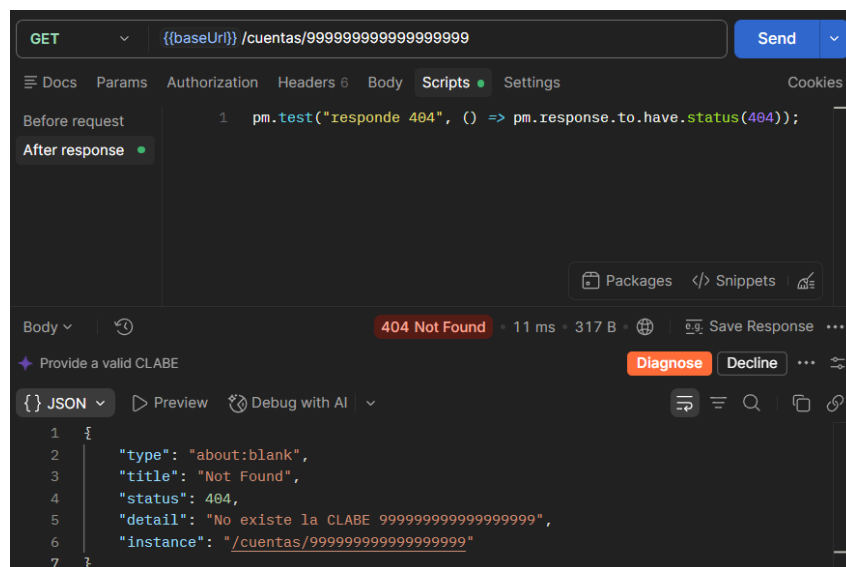


Figura 6. Consulta de cuenta que no existe.

4.- Persistencia de datos con y sin volumen

Primero se ejecutó `docker compose down`, seguido de `docker compose up -d`, y posteriormente se volvió a consultar la información mediante Postman. Los datos registrados previamente permanecieron disponibles después de recrear los contenedores.

Esto se debe a que `docker compose down` elimina los contenedores y otros recursos asociados, pero no elimina el volumen utilizado para almacenar los datos de

PostgreSQL. Por lo tanto, al crear nuevamente el contenedor de la base de datos, este puede utilizar los datos que permanecieron almacenados en el volumen.

Posteriormente se ejecutó `docker compose down -v`, que además de eliminar los contenedores elimina los volúmenes asociados al proyecto. Después de levantar nuevamente los servicios y realizar la misma consulta mediante Postman, los datos registrados anteriormente ya no se encontraban disponibles.

Esta prueba permite comprobar la importancia de utilizar almacenamiento persistente para una base de datos: la vida útil del contenedor no tiene que determinar la vida útil de los datos. El volumen permite separar la persistencia de la información del ciclo de vida del contenedor.

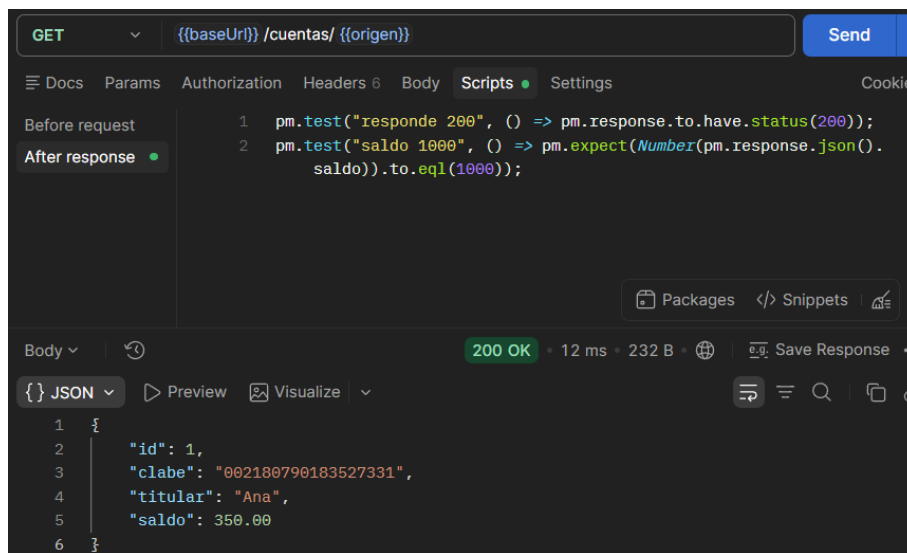


Figura 7. Consulta de cuenta antes y después de `Docker compose down` y `Docker compose up -d`.

```
PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1\mexi-banco> docker compose down
[+] down 3/3
✓Container mexi-banco-app-1 Removed 0.5s
✓Container mexi-banco-db-1 Removed 0.3s
✓Network mexibanco Removed 0.3s
PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1\mexi-banco> docker compose up -d
[+] up 3/3
✓Network mexibanco Created 0.0s
✓Container mexi-banco-db-1 Healthy 5.9s
✓Container mexi-banco-app-1 Started 6.0s
PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1\mexi-banco> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS
mexi-banco-app-1 mexi-banco-app "java -jar app.jar" app 39 seconds ago Up 33 seco
nds (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1 postgres:17-alpine "docker-entrypoint.s..." db 39 seconds ago Up 38 seco
nds (healthy) 5432/tcp
```

Figura 8. Eliminando los contenedores y levantándolos de nuevo sin eliminar el volumen.

```

PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1\mexi-banco> docker compose down -v
[+] down 4/4
✓ Container mexi-banco-app-1      Removed      0.3s
✓ Container mexi-banco-db-1      Removed      0.2s
✓ Volume mexi-banco_mexibanco_datos Removed      0.0s
✓ Network mexibanco              Removed      0.3s
PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1\mexi-banco> docker compose up -d
[+] up 4/4
✓ Network mexibanco              Created      0.0s
✓ Volume mexi-banco_mexibanco_datos Created      0.0s
✓ Container mexi-banco-db-1      Healthy     5.8s
✓ Container mexi-banco-app-1      Started     6.0s
PS C:\Users\mauri\Documents\Universidad\SDistribuidos\SD-2027-1\mexi-banco> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS
mexi-banco-app-1    mexi-banco-app      "java -jar app.jar"    app        19 seconds ago Up 12 seco
nds (healthy)      0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine  "docker-entrypoint.s... db          19 seconds ago Up 17 seco
nds (healthy)      5432/tcp

```

Figura 9. Eliminando los contenedores y volumen.

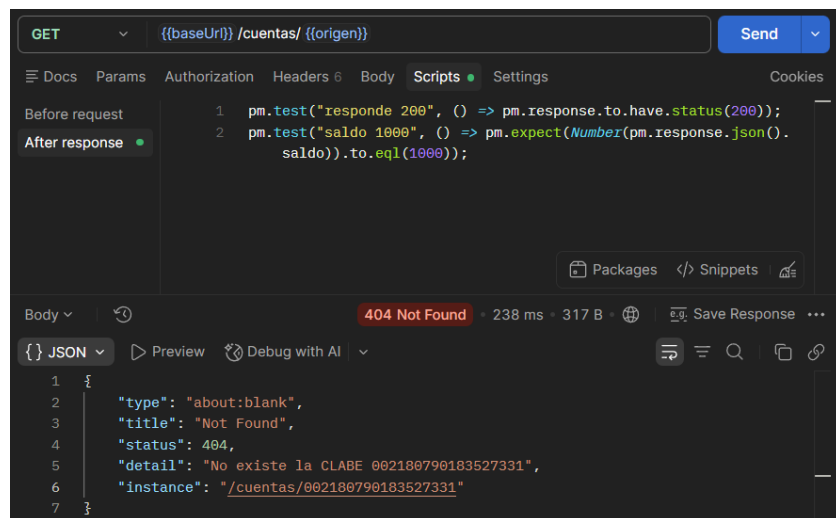


Figura 10. Consulta de cuenta después de levantar contenedores habiendo eliminado el volumen.

En consecuencia, `down` y `down -v` producen efectos diferentes sobre la información almacenada. El primero permite recrear los contenedores conservando los datos, mientras que el segundo elimina también el almacenamiento persistente asociado.